

# Color Block Tracking

## 1. Function Description

After the program starts, press [r], [g], [b], [y] keys to select the color block to track. Press [c] to cancel tracking. At this point, you can press [r], [g], [b], [y] to change to other target color blocks.

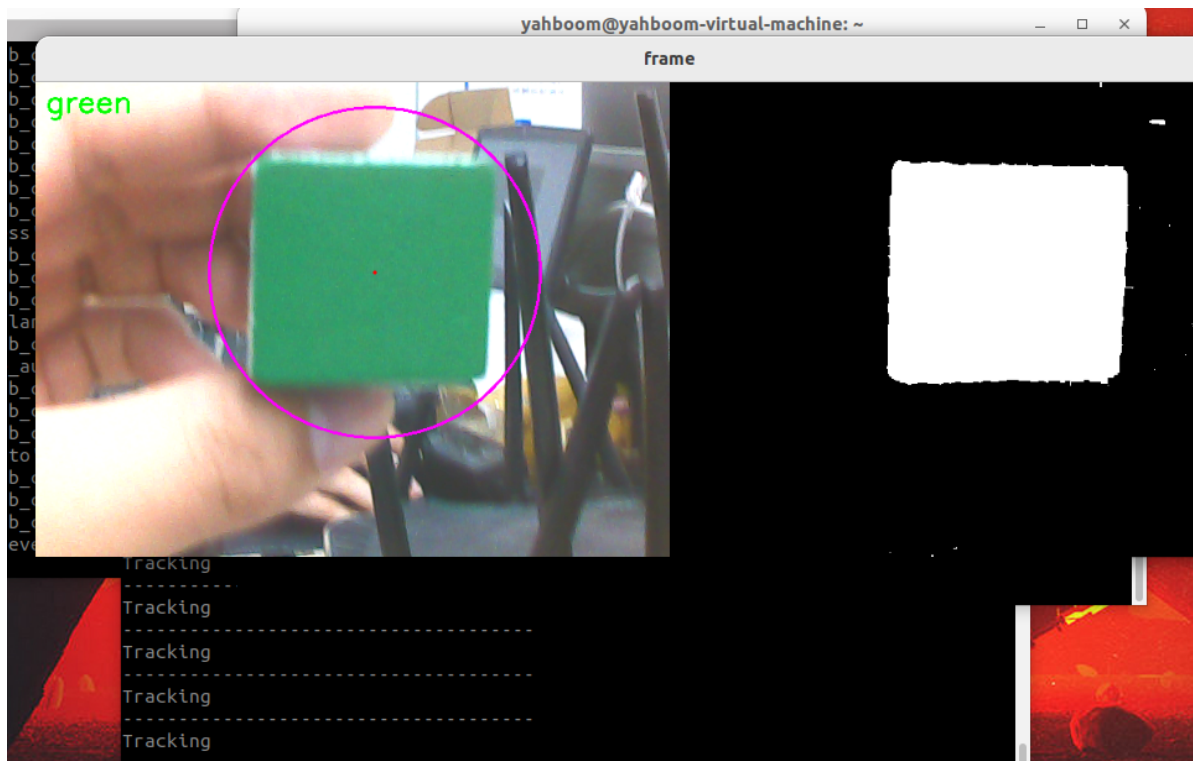
## 2. Startup and Operation

Enter the following commands in the terminal to start:

**Note: When tracking color blocks, move slowly within the camera's field of view**

```
# Start camera and inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start robotic arm control program
ros2 run dofbot_sorting_3d grasp
# Start color block tracking program
ros2 run dofbot_follow color_follow_2D
```

Taking the green block as an example, press [g] to select the green block, then press the spacebar to start tracking.



If you want to change to the yellow block at this point, press [c] first to cancel the color block selection, then press [y] to select the yellow block.

## 3. Core Code Analysis

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_follow/dofbot_follow/color_follow_2D.py
```

Import necessary libraries:

```
import cv2
import os
import numpy as np
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2 as cv
from dofbot_sorting_3d.color_common import *
from std_msgs.msg import Bool, Int16
import time
from rclpy.node import Node
import rclpy
import threading
from Arm_Lib import Arm_Device
from dofbot_interface.msg import AprilTagInfo
import dofbot_follow.PID as PID
```

Initialize program parameters, create publishers and subscribers:

```
class ColorRecognizeNode(Node):
    # Track color blocks

    def __init__(self, name):
        # Initialize the ROS2 node with the given name
        # 使用给定的名称初始化ROS2节点
        super().__init__(name)

        # Create an instance of the robotic arm device class
        # 创建机械臂设备类的实例
        self.Arm = Arm_Device()

        # Define initial joint angles for the robot arm
        # 定义机械臂的初始关节角度
        self.init_joints = [90, 150, 0, 20, 90, 0]

        # Bridge for converting between OpenCV images and ROS2 Image messages
        # 用于在OpenCV图像和ROS2 Image消息之间转换的桥接器
        self.rgb_bridge = CvBridge()

        # Flag indicating whether to publish position information
        # 标志位，表示是否发布位置信息
        self.pub_pos_flag = False

        # Publisher for publishing object position information
        # 发布物体位置信息的发布者
        self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo", 1)

        # Publisher for setting joint angle
        # 设置关节角度的发布者
        self.TargetJoint5_pub = self.create_publisher(Int16, "set_joint5", 10)

        # Current target color
        # 当前目标颜色
        self.target_color = 0

        # File paths for HSV color threshold configuration files
        # HSV颜色阈值配置文件的路径
```

```
self.red_hsv_text = os.path.join(package_pwd, 'red_colorHSV.text')
self.green_hsv_text = os.path.join(package_pwd, 'green_colorHSV.text')
self.blue_hsv_text = os.path.join(package_pwd, 'blue_colorHSV.text')
self.yellow_hsv_text = os.path.join(package_pwd, 'yellow_colorHSV.text')

# Tuple to store HSV range for color filtering
# 存储颜色过滤的HSV范围的元组
self.hsv_range = ()

# Flag for selection state (e.g., selecting ROI)
# 选择状态标志
self.select_flags = False

# Name of the display window
# 显示窗口的名称
self.windows_name = 'frame'

# State machine state for tracking
# 跟踪的状态机状态
self.Track_state = 'init'

# Mouse click position coordinates
# 鼠标点击位置坐标
self.Mouse_XY = (0, 0)

# Image dimensions (columns, rows)
# 图像尺寸（列，行）
self.cols, self.rows = 0, 0

# Initial region of interest (ROI) for color detection
# 颜色检测的初始感兴趣区域（ROI）
self.Roi_init = ()

# Instance of color detection class
# 颜色检测类的实例
self.color = color_detect()

# Currently detected color
# 当前检测到的颜色
self.cur_color = None

# Text color for display
# 显示的文本颜色
self.text_color = (0, 0, 0)

# Center coordinates of detected object
# 检测到的物体中心坐标
self.cx = 0
self.cy = 0

# Radius of detected circular object
# 检测到的圆形物体的半径
self.circle_r = 0

# Message for publishing joint angle
# 发布关节角度的消息
self.joint5 = Int16()
```

```

# Array to store corner points of detected object
# 存储检测到的物体角点的数组
self.corners = np.empty((4, 2), dtype=np.int32)

# Current target color index
# 当前目标颜色索引
self.cur_target_color = 0

# Flag for updating HSV thresholds
# 更新HSV阈值的标志
self.updata_flag = False

# Move arm to initial position on startup
# 启动时将机械臂移动到初始位置
self.Arm.Arm_serial_servo_write6_array(self.init_joints, 2000)

# Default distance parameter
# 默认距离参数
self.dist = 0.13

# Subscription to receive raw camera images
# 订阅以接收原始相机图像
self.subscription = self.create_subscription(
    Image, '/image_raw', self.ImageCallback, 1)

# Variables for storing object position
# 存储物体位置的变量
self.px = 0.0
self.py = 0.0

# Target servo positions for x and y axes
# x轴和y轴的目标舵机位置
self.target_servox = 90
self.target_servoy = 45

# PID controllers for x and y axis servo control
# x轴和y轴舵机控制的PID控制器
#self.xservo_pid = PID.PositionalPID(0.25, 0.1, 0.05)
#self.yservo_pid = PID.PositionalPID(0.25, 0.1, 0.05)
self.xservo_pid = PID.PositionalPID(0.5, 0.1, 0.1)
self.yservo_pid = PID.PositionalPID(0.5, 0.1, 0.1)

# Flags indicating if servo output is out of range
# 标志位，表示舵机输出是否超出范围
self.y_out_range = False
self.x_out_range = False

# Current joint angles of the arm
# 机械臂当前的关节角度
self.cur_joints = self.init_joints

# Auxiliary flags for tracking logic
# 跟踪逻辑的辅助标志
self.a = 0
self.b = 0

```

ImageCallback callback function:

```

def Imagecallback(self, color_frame):
    # Convert ROS2 Image message to OpenCV RGB image format
    # 将ROS2 Image消息转换为OpenCV RGB图像格式
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')

    # Convert RGB image to BGR format for OpenCV processing
    # 将RGB图像转换为BGR格式以进行OpenCV处理
    rgb_image = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)

    # Create a copy of the image for drawing results
    # 创建图像副本用于绘制结果
    result_image = np.copy(rgb_image)

    # Capture key press
    # 捕获按键
    key = cv2.waitKey(10) & 0xFF

    # Process the image to detect colors and get binary mask
    # 处理图像以检测颜色并获取二值掩码
    result_frame, binary = self.process(rgb_image, key)

    # Start a separate thread to display the image
    # 启动独立线程显示图像
    # This prevents blocking the main callback loop
    # 这可以防止阻塞主回调循环
    show_frame = threading.Thread(target=self.img_out, args=(result_frame,
binary,))
    show_frame.start()
    show_frame.join() # wait for the display thread to finish

    # Check if a valid object (with sufficient size) has been detected
    # 检查是否检测到有效物体
    # center coordinates non-zero, radius > 30 pixels
    # 中心坐标非零, 半径 > 30像素
    if self.cx != 0 and self.cy != 0 and self.circle_r > 30:
        # Get the center coordinates of the detected object
        # 获取检测到的物体中心坐标
        center_x = self.cx
        center_y = self.cy

        # Check if the object center deviates significantly from image center
        # 检查物体中心是否明显偏离图像中心
        # Image center: (320, 240) for 640x480 resolution
        # 图像中心: (320, 240), 对应640x480分辨率
        # Threshold: 10 pixels (dead zone to avoid jitter)
        # 阈值: 10像素 (死区以避免抖动)
        if (abs(center_x - 320) > 10 or abs(center_y - 240) > 10):
            # Call XY tracking function to adjust arm position
            # 调用XY跟踪函数以调整机械臂位置
            self.XY_track(center_x, center_y)

            # Print tracking status
            # 打印跟踪状态
            print("Tracking")
            print("-----")

```

Image processing function process:

```

def process(self, rgb_img, key):

    rgb_img = cv.resize(rgb_img, (640, 480))

    # Initialize empty binary image list
    # 初始化空的二值图像列表
    binary = []

    # Key press handling for color selection and mode switching
    # 按键处理，用于颜色选择和模式切换
    if key == ord('c') or key == ord('C'):
        # 'C' key: Cancel/Reset - clear target color and reset system
        # 'C'键：取消/重置
        self.target_color = 0
        self.Reset()
        self.update_flag = True
    elif key == ord('r') or key == ord('R'):
        # 'R' key: Select Red color
        # 'R'键：选择红色
        self.target_color = 1
        self.cur_target_color = self.target_color
    elif key == ord('g') or key == ord('G'):
        # 'G' key: Select Green color
        # 'G'键：选择绿色
        self.target_color = 2
        self.cur_target_color = self.target_color
    elif key == ord('b') or key == ord('B'):
        # 'B' key: Select Blue color
        # 'B'键：选择蓝色
        self.target_color = 3
        self.cur_target_color = self.target_color
    elif key == ord('y') or key == ord('Y'):
        # 'Y' key: Select Yellow color
        # 'Y'键：选择黄色
        self.target_color = 4
        self.cur_target_color = self.target_color
    elif key == ord('i') or key == ord('I') or self.target_color != 0:
        # 'I' key or any color selected: Enter identify/tracking state
        # 'I'键或选择了任何颜色：进入识别/跟踪状态
        self.Track_state = "identify"
    # print("self.Track_state: ", self.Track_state)

    # State machine: Initialization state
    # 状态机：初始化状态
    if self.Track_state == 'init':
        # Create a named window for mouse interaction
        # 创建用于鼠标交互的命名窗口
        cv.namedWindow(self.windows_name, cv.WINDOW_AUTOSIZE)
        # Set mouse callback for selecting ROI
        # 设置鼠标回调以选择感兴趣区域
        cv.setMouseCallback(self.windows_name, self.onMouse, 0)

    # If ROI selection is active, draw guides on the image
    # 如果ROI选择处于活动状态，在图像上绘制引导线
    if self.select_flags == True:
        # Draw line from start to current mouse position
        # 从起点到当前鼠标位置绘制线条

```

```

        cv.line(rgb_img, self.cols, self.rows, (255, 0, 0), 2)
        # Draw rectangle for ROI
        # 为ROI绘制矩形
        cv.rectangle(rgb_img, self.cols, self.rows, (0, 255, 0), 2)

        # If ROI is valid (width and height > 0)
        # 如果ROI有效 (宽度和高度 > 0)
        if self.Roi_init[0] != self.Roi_init[2] and self.Roi_init[1] !=
self.Roi_init[3]:
            # Extract HSV range from the selected ROI
            # 从选定的ROI中提取HSV范围
            rgb_img, self.hsv_range = self.color.Roi_hsv(rgb_img,
self.Roi_init)
            self.dyn_update = True
        else:
            # Invalid ROI, return to init state
            # ROI无效, 返回初始化状态
            self.Track_state = 'init'

# State machine: Identify/Tracking state
# 状态机: 识别/跟踪状态
elif self.Track_state == "identify":
    # Load HSV range based on selected color
    # 根据选择的颜色加载HSV范围
    if self.target_color == 1:
        self.hsv_range = read_HSV(self.red_hsv_text)
        self.cur_color = "red"
        self.text_color = (0, 0, 255) # Red text (BGR)
    elif self.target_color == 2:
        self.hsv_range = read_HSV(self.green_hsv_text)
        self.cur_color = "green"
        self.text_color = (0, 255, 0) # Green text (BGR)
    elif self.target_color == 3:
        self.hsv_range = read_HSV(self.blue_hsv_text)
        self.cur_color = "blue"
        self.text_color = (255, 0, 0) # Blue text (BGR)
    elif self.target_color == 4:
        self.hsv_range = read_HSV(self.yellow_hsv_text)
        self.cur_color = "yellow"
        self.text_color = (255, 255, 0) # Yellow text (BGR)
    else:
        # No valid color selected, return to init
        # 未选择有效颜色, 返回初始化状态
        self.Track_state = 'init'

# Object detection and tracking
# 物体检测与跟踪
if self.Track_state != 'init':
    # Ensure HSV range is loaded before detection
    # 确保在检测前已加载HSV范围
    if len(self.hsv_range) != 0:

        # Returns: annotated image, binary mask, circle params, _, corners
        # 返回: 标注图像, 二值掩码, 圆形参数, _, 角点
        rgb_img, binary, self.circle, _, self.corners =
self.color.object_follow(rgb_img, self.hsv_range)

        # Extract circle parameters

```

```

# 提取圆形参数
self.cx = self.circle[0] # Center x
self.cy = self.circle[1] # Center y
self.circle_r = self.circle[2] # Radius

# Update HSV configuration file if flag is set
# 如果标志已设置，则更新HSV配置文件
if self.cur_target_color == 1 and self.updata_flag == True:
    write_HSV(self.red_hsv_text, self.hsv_range)
elif self.cur_target_color == 2 and self.updata_flag == True:
    write_HSV(self.green_hsv_text, self.hsv_range)
elif self.cur_target_color == 3 and self.updata_flag == True:
    write_HSV(self.blue_hsv_text, self.hsv_range)
elif self.cur_target_color == 4 and self.updata_flag == True:
    write_HSV(self.yellow_hsv_text, self.hsv_range)
self.updata_flag = False # Reset update flag

# Overlay the current color name on the image
# 在图像上叠加当前颜色名称
...

```

XY\_track function:

```

def XY_track(self, center_x, center_y):
    # Store the current detected object's center position
    # 存储当前检测到的物体中心位置
    self.px = center_x
    self.py = center_y

    # X-axis Servo Control
    # X轴舵机控制
    if not (self.target_servox >= 180 and center_x <= 320 and self.a == 1 or
            self.target_servox <= 0 and center_x >= 320 and self.a == 1):
        # Normal tracking mode
        # 正常跟踪模式
        if self.a == 0:
            # Set the current object's x-coordinate as the PID system output
            # 将当前物体的x坐标设为PID系统输出
            self.xservo_pid.SystemOutput = center_x

            # Handle out-of-range situation
            # 处理超出范围的情况
            if self.x_out_range == True:
                # Force the servo to the boundary position
                # 强制舵机到达边界位置
                if self.target_servox < 0:
                    self.target_servox = 0
                    self.xservo_pid.SetStepSignal(630)
                if self.target_servox > 0:
                    self.target_servox = 180
                    self.xservo_pid.SetStepSignal(10)
                self.x_out_range = False
            else:
                # set the step signal to the image center x (320)
                # 将步进信号设为图像中心x坐标 (320)
                self.xservo_pid.SetStepSignal(320)
                self.x_out_range = False

```



```

# Set the inertia time parameters for the PID controller
# 设置PID控制器的惯性时间参数
self.xservo_pid.SetInertiaTime(0.01, 0.1)

# Calculate the target value for the servo
# 计算舵机的目标值
target_valuex = int(1500 + self.xservo_pid.SystemOutput)

# Convert the target value to a servo angle
# 将目标值转换为舵机角度
self.target_servox = int((target_valuex - 500) / 10) - 10

# Check if the calculated angle exceeds the valid range
# 检查计算出的角度是否超出有效范围
if self.target_servox > 180:
    self.x_out_range = True
if self.target_servox < 0:
    self.x_out_range = True

#Y-axis Servo Control
#Y轴舵机控制

if not (self.target_servoy >= 180 and center_y <= 240 and self.b == 1 or
        self.target_servoy <= 0 and center_y >= 240 and self.b == 1):
    if self.b == 0:
        # Set the current object's y-coordinate as the PID system output
        # 将当前物体的y坐标设为PID系统输出
        self.y servo_pid.SystemOutput = center_y

        # Handle out-of-range situation
        # 处理超出范围的情况
        if self.y_out_range == True:
            self.y servo_pid.SetStepSignal(450)
            self.y_out_range = False
        else:
            # set the step signal to the image center y (240)
            # 将步进信号设为图像中心y坐标 (240)
            self.y servo_pid.SetStepSignal(240)

    # Set the inertia time parameters for the PID controller
    # 设置PID控制器的惯性时间参数
    self.y servo_pid.SetInertiaTime(0.01, 0.1)

    # Calculate the target value for the servo
    # 计算舵机的目标值
    target_valuey = int(1500 + self.y servo_pid.SystemOutput)

    # Enforce a minimum value to prevent the arm from going too low
    # 强制执行最小值，防止机械臂降得太低
    if target_valuey <= 1000:
        target_valuey = 1000
        self.y_out_range = True

    # Convert the target value to a servo angle with an offset
    # 将目标值转换为带偏移量的舵机角度
    self.target_servoy = int((target_valuey - 500) / 10) - 55

```

```

        # Clamp the angle to the valid range [0, 180]
        # 将角度限制在有效范围[0, 180]内
        if self.target_servoy > 180:
            self.target_servoy = 180
        if self.target_servoy < 0:
            self.target_servoy = 0

        #Map servo angles to physical joint angles
        #舵机角度映射到物理关节角度
        joint2 = 120 + self.target_servoy
        joint3 = self.target_servoy / 4.5
        joint4 = self.target_servoy / 3

    joints_0 = [float(self.target_servox / 1), float(joint2), float(joint3),
float(joint4), 90.0, 30.0]

    # Send the joint commands to the robotic arm
    # 向机械臂发送关节控制命令
    self.Arm.Arm_serial_servo_write6_array(joints_0, 2000)

    # Update the current joint state
    # 更新当前关节状态
    self.cur_joints = joints_0

```